

数据库系统原理

胡 敏 教授 博士生导师
合肥工业大学 计算机与信息学院
jsjxhumin@hfut.edu.cn

第3章 关系数据库标准语言SQL



厚德 笃学 崇实 尚新

3.4 查 询

3.4.1 单表查询

3.4.2 连接查询

3.4.3 嵌套查询

3.4.4 集合查询

3.4.5 基于派生表的查询

3.4.6 **SELECT** 语句的一般格式



3.4.3 嵌套查询

- 嵌套查询概述
- 嵌套查询分类
- 嵌套查询求解方法
- 引出子查询的谓词



3.4.3 嵌套查询（子查询）

- 一个SELECT-FROM-WHERE语句称为一个查询块。
- 将一个查询块嵌套在另一个查询块的WHERE子句或HAVING短语的条件中，称为子查询。
- 子查询中不能使用ORDER BY子句。
- 不相关子查询与相关子查询的区别与处理。

SELECT Sname

外层查询/父查询

FROM Student

WHERE Sno IN

(SELECT Sno

内层查询/子查询

FROM SC

WHERE Cno= '2') ;

- 层层嵌套方式反映了 SQL语言的结构化，是SQL的亮点之一。

- 子查询返回一个集合，可以通过与这个集合的比较来确定另一个查询集合。
- 三种类型的子查询：(NOT) IN-子查询； θ -Some/ θ -All子查询；(NOT) EXISTS子查询



子查询

为什么需要子查询？

- 现实中，很多情况需要进行下述条件的判断
 - ✓ 集合成员资格
某一元素是否是某一个集合的成员
 - ✓ 集合之间的比较
 - ✓ 某一个集合是否包含另一个集合等
- 集合基数的测试
 - ✓ 测试集合是否为空
 - ✓ 测试集合是否存在重复元组



嵌套查询分类

■ 不相关子查询

子查询的查询条件不依赖于父查询。是由里向外逐层处理。即每个子查询在上一级查询处理之前求解，子查询的结果用于建立其父查询的查找条件。

■ 相关子查询

子查询的查询条件依赖于父查询

- 首先取外层查询中表的第一个元组，根据它与内层查询相关的属性值处理内层查询，若**WHERE**子句返回值为真，则取此元组放入结果表；
- 然后再取外层表的下一个元组；
- 重复这一过程，直至外层表全部检查完为止。



引出子查询的谓词

- 带有IN谓词的子查询
- 带有比较运算符的子查询
- 带有ANY或ALL谓词的子查询
- 带有EXISTS谓词的子查询



带有IN谓词的子查询

■ 基本语法:

表达式 [*not*] *in* (子查询)

- 语法中，表达式的最简单形式就是列名或常数。
- 语义：判断某一表达式的值是否在子查询的结果中。



带有IN谓词的子查询

[例32] 查询与“刘晨”在同一个系学习的学生。此查询要求可以分步来完成

① 确定“刘晨”所在系名。

```
SELECT Sdept FROM Student WHERE Sname= '刘晨'; // 结果为: CS
```

② 查找所有在系学习的学生。

```
SELECT Sno, Sname, Sdept FROM Student WHERE Sdept= 'CS';
```

③ 用嵌套查询实现:

SELECT S 用自身连接完成本查询的要求

```
WHERE      SELECT S1.Sno, S1.Sname, S1.Sdept
            FROM Student S1, Student S2
            WHERE S1.Sdept = S2.Sdept AND
                  S2.Sname = '刘晨';
```

此查询为不相关子查询。DBMS求解该查询时也是分步去做的。

$$\pi_{Sno, Sname, Sdept}(\sigma_{Sdept=(\pi_{Sdept}(\sigma_{Sname='刘晨'}(Student)))} \wedge Sname \neq '刘晨'}(Student))$$

关系代数表达式?



带有IN谓词的子查询

[例33] 查询选修了“信息系统”的学生学号和姓名。

```
SELECT Sno, Sname  
FROM Student  
WHERE Sno IN
```

③ 最后在Student关系中
取出Sno和Sname

```
(SELECT Sno  
FROM SC  
WHERE Cno =  
(SELECT Cno  
FROM Course  
WHERE Cname = '信息系统')  
);
```

用连接查询

```
SELECT Sno, Sname  
FROM Student, SC, Course  
WHERE Student.Sno = SC.Sno AND  
SC.Cno = Course.Cno AND  
Course.Cname='信息系统';
```

带有比较运算符的子查询

- 当能确切知道内层查询返回单值时，可用比较运算符（>，<，=，>=，<=，!=或<>）。

例：假设一个学生只可能在一个系学习，并且必须属于一个系，则在[例31]可以用 = 代替IN：

```
SELECT Sno, Sname, Sdept FROM Student
WHERE Sdept =      // 结果为单值时，可用=代替in
      (SELECT Sdept FROM Student
       WHERE Sname= '刘晨' );
```



带有比较运算符的子查询

例：求既学过1号课程, 又学过2号课程的学生学号

- **Select Sno From SC Where Cno = '1' and**
- **Sno in (Select Sno From SC Where Cno = '2')**

Select S1.Sno From SC s1,sc s2 Where s1.sno=s2.sno and s1.cno='1' and s2.cno='2'

还能够怎样书写?

? : 找出每个学生超过他自己选修课程平均成绩的课程号

Select Sno, Cno

Sno	Cno	Grade
202215121	1	92
202215121	2	85
202215121	3	88
202215122	2	90
202215122	3	80
202215126	1	(NULL)

**(select avg(Grade)
from SC y
where y.Sno=x.Sno);**

Sno	Cno
202215121	1
202215122	2

相关子查询



3.4.3 嵌套查询

- 1.带有**IN**谓词的子查询
- 2.带有比较运算符的子查询
- 3.带有**ANY**或**ALL**谓词的子查询
- 4.带有**EXISTS**谓词的子查询



带有**ANY**或**ALL**谓词的子查询

- 在谓词逻辑中，还有存在量词和全称量词的概念，在**SQL**中并没有对应的表达，统一采用“谓词”来表达
 - 方法一：引入**ANY**和**ALL**谓词，其对象为某个查询结果，表示其中任意一个值或者全部值
 - 方法二：引入**EXIST**谓词，其对象也是某个查询结果，但表示这个查询结果是否为空，返回真值。

■ 谓词语义

- **ANY (some)**：任意一个值
- **ALL**：所有值

注：**SQL标准中**，由于其语义的模糊性：**any**，“任一”是指所有呢？还是指某一个？不清楚，所以被 **some** 替代以求更明晰。



带有ANY或ALL谓词的子查询

- 在谓词逻辑中，还有存在量词和全称量词的概念，在SQL中并没有对应的表达，统一采用“谓词”来表达

- 方法一：

- ★ 引入ANY (some)：任意一个值
- ★ 引入ALL：所有值

- 方法二：引入EXIST谓词，其对象也是某个查询结果，但表示这个查询结果是否为空，返回真值。

- 基本语法：

- 表达式 θany (子查询)

- 表达式 θall (子查询)

- ✓ 语法中，是比较运算符：< , > , >= , <= , = , <>。

- ✓ 语义：将表达式的值与子查询的结果进行比较：

- 如果表达式的值至少与子查询结果的某一个值相比较满足 θ 关系，则“表达式 θany (子查询)”的结果便为真；

- 如果表达式的值与子查询结果的所有值相比较都满足 θ 关系，则“表达式 θall (子查询)”的结果便为真。

注：SQL标准中，由于其语义的模糊性：any，“任一”是指所有呢？还是指某一个？不清楚，所以被 some 替代以求更明晰。

与“某一个”比较，还是与“所有”比较？



带有ANY或ALL谓词的子查询

■ 需要配合使用比较运算符(ANY在有些版本中用SOME)语义

> ANY	大于子查询结果中的某个值
> ALL	大于子查询结果中的所有值
< ANY	小于子查询结果中的某个值
< ALL	小于子查询结果中的所有值
>= ANY	大于等于子查询结果中的某个值
>= ALL	大于等于子查询结果中的所有值
<= ANY	小于等于子查询结果中的某个值
<= ALL	小于等于子查询结果中的所有值
= ANY	等于子查询结果中的某个值
= ALL	等于子查询结果中的所有值 (通常没有实际意义)
!= (或<>) ANY	不等于子查询结果中的某个值
!= (或<>) ALL	不等于子查询结果中的任何一个值



带有ANY或ALL谓词的子查询（续）

[例34] 查询其他系中比信息系任意一个(其中某一个)学生年龄小的学生姓名和年龄。

```
SELECT Sname, Sage FROM Student
WHERE Sage < ANY (SELECT Sage
                  FROM Student
                  WHERE Sdept= ' CS ')
```

Sno	Sname	Sdept	Ssex	Sage
202215121	李勇	CS	男	20
202215122	刘晨	CS	女	19
202215123	王敏	MA	女	18
202215125	张立	IS	男	19
202215126	欧阳杰	AI	男	18

结果

Sname	Sage
王敏	18
张立	19
欧阳杰	18

执行过程

- 1.DBMS执行此查询时，首先处理子查询，找出CS系中所有学生的年龄，构成一个集合(20, 19)
2. 处理父查询，找所有不是CS系且年龄小于20 或 19的学生



带有ANY或ALL谓词的子查询

■ ANY和ALL谓词有时可以用聚集函数实现

ANY与ALL与集函数的对应关系如下表：

	=	<>或!=	<	<=	>	>=
ANY	IN	--	<MAX	<=MAX	>MIN	>= MIN
ALL	--	NOT IN	<MIN	<= MIN	>MAX	>= MAX

- ✓ 用聚集函数实现子查询通常比直接用ANY或ALL查询效率高，因为前者通常能够减少比较次数。



带有ANY或ALL谓词的子查询

[例35] 用聚集函数实现[例34]。

```
SELECT Sname, Sage
FROM Student
WHERE Sage <
      (SELECT MAX(Sage)
       FROM Student
       WHERE Sdept= ' CS ')
AND Sdept <> 'CS';
```

Sname	Sage
王敏	18
张立	19
欧阳杰	18



带有ANY或ALL谓词的子查询

[例36] 查询其他系中比信息系所有学生年龄都小的学生姓名及年龄。

方法一：用ALL谓词

```
SELECT Sname, Sage FROM Student
WHERE Sage < ALL (SELECT Sage FROM Student WHERE Sdept= ' IS ')
AND Sdept <> ' IS ';
```

方法二：用聚集函数

```
SELECT Sname, Sage FROM Student
WHERE Sage < (SELECT MIN(Sage) FROM Student WHERE Sdept= ' IS ')
AND Sdept <> ' IS ';
```



带有EXISTS谓词的子查询

■ 基本语法:

■ *[not] Exists* (子查询)

➤ 语义: 子查询结果中是否有元组存在

■ EXISTS谓词

◆ 存在量词 $\exists$

◆ 带有EXISTS谓词的子查询不返回任何数据, 只产生逻辑真值“true”或逻辑假值“false”。

➤ 若内层查询结果非空, 则外层的WHERE子句返回真值

➤ 若内层查询结果为空, 则外层的WHERE子句返回假值

◆ 由EXISTS引出的子查询, 其目标列表表达式通常都用*, 因为带EXISTS的子查询只返回真值或假值, 给出列名无实际意义。

■ NOT EXISTS谓词



带有EXISTS谓词的子查询

■ EXISTS谓词

◆ 存在量词 $\exists$

◆ 带有EXISTS谓词的子查询不返回任何数据，只产生逻辑真值“true”或逻辑假值“false”。

➤ 若内层查询结果非空，则外层的WHERE子句返回真值

➤ 若内层查询结果为空，则外层的WHERE子句返回假值

◆ 由EXISTS引出的子查询，其目标列表表达式通常都用*，因为带EXISTS的子查询只返回真值或假值，给出列名无实际意义。

■ NOT EXISTS谓词



带有EXISTS谓词的子查询

[例37] 查询所有选修了2号课程的学生姓名。

思路分析：

- 本查询涉及Student和SC关系；
- 在Student中依次取每个元组的Sno值，用此值去检查SC关系；
- 若SC中存在这样的元组，其Sno值等于此Student.Sno值，并且其Cno= '2'，则取此Student.Sname送入结果关系。

✓ 用嵌套查询：

```
SELECT Sname FROM Student WHERE EXISTS (SELECT *  
      FROM SC WHERE Sno=Student.Sno AND Cno= '2');
```

✓ 用连接查询：

```
SELECT Sname FROM Student, SC  
      WHERE Student.Sno=SC.Sno AND SC.Cno= '2';
```

不加not形式的Exists谓词可以不用



带有EXISTS谓词的子查询

[例38] 查询没有选修2号课程的学生姓名。

```
SELECT Sname FROM Student
WHERE NOT EXISTS
  (SELECT * FROM SC
   WHERE Sno = Student.Sno AND Cno='2');
```

Sname
张立
欧阳杰
王敏



带有EXISTS谓词的子查询(续)

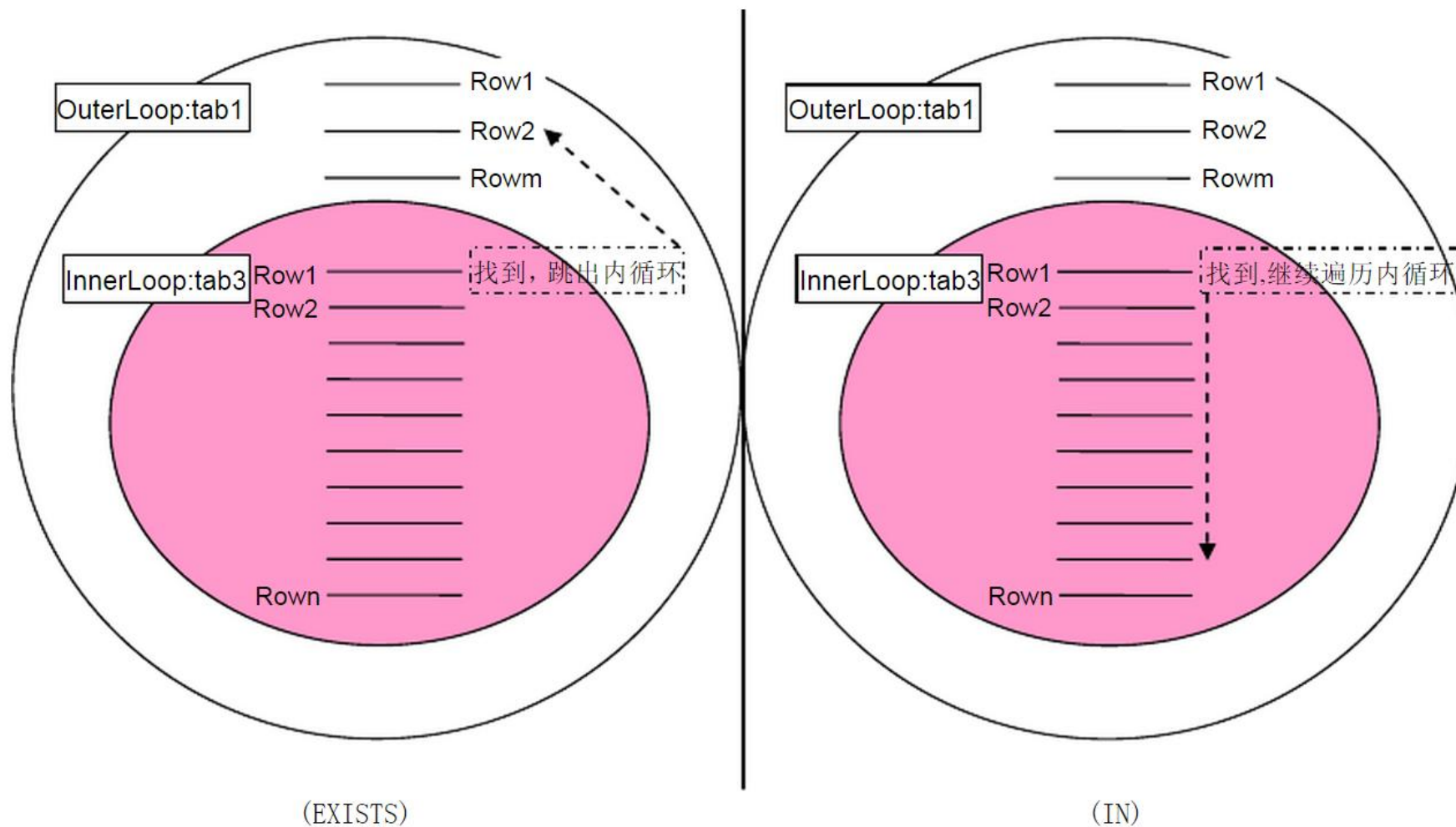
■ 不同形式的查询间的替换

- ✓ 一些带EXISTS或NOT EXISTS谓词的子查询不能被其他形式的子查询等价替换
- ✓ 所有带IN谓词、比较运算符、ANY和ALL谓词的子查询都能用带EXISTS谓词的子查询等价替换。



带有EXISTS谓词的子查询(续)

为什么在嵌套连接中，EXISTS会比IN效率高？



带有EXISTS谓词的子查询

■ 用EXISTS/NOT EXISTS实现全称量词(难点)

- SQL语言中没有全称量词 $\forall$ (For all)
- 可以把带有全称量词的谓词转换为等价的带有存在量词的谓词:

用EXISTS实现逻辑蕴含

$$(\forall x)P \equiv \neg (\exists x(\neg P))$$



不同形式的查询间的替换

[例39] 查询选修了全部课程的学生姓名。

SQL语句写法: 查询这样的学生, 没有一门课程是他不选修的

```
SELECT Sname FROM Student
WHERE NOT EXISTS
  (SELECT * FROM Course
   WHERE NOT EXISTS
    (SELECT * FROM SC
     WHERE Sno= Student.Sno AND Cno= Course.Cno ) ) ;
```

//不存在

//有一门课程

//学生没有选修该课程

■ 利用关系代数 怎样表达?

$\pi_{Sname} (Student \bowtie (\pi_{Sno,Cno}(SC) \div \pi_{Cno}(Course)))$

■ 利用元组演算 怎样表达?

GET W (Student.Sname): $\forall CX \exists SCX (SCX.Sno=Student.Sno \wedge SCX.Cno=CX.Cno)$



用EXISTS/NOT EXISTS实现逻辑蕴涵

■ 用EXISTS/NOT EXISTS实现逻辑蕴涵(难点)

- ◆ SQL语言中没有逻辑蕴涵运算。
- ◆ 可以利用谓词演算将逻辑蕴涵谓词等价转换为：

$$p \rightarrow q \equiv \neg p \vee q$$

p	q	$P \rightarrow q$	$\neg p$	q	$\neg p \vee q$
0	0	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	0
1	1	1	0	1	1



3.4.3 嵌套查询

[例40] 查询至少选修了学生202215122选修的全部课程的学生号码。

解题思路：

- 用逻辑蕴涵表达：查询学号为x的学生，对所有的课程y，只要202215122学生选修了课程y，则x也选修了y。
- 形式化表示：
用p表示谓词 “学生202215122选修了课程y”
用q表示谓词 “学生x选修了课程y”
则上述查询为： $(\forall y) p \rightarrow q$
- 等价变换：
$$\begin{aligned}(\forall y)p \rightarrow q &\equiv \neg (\exists y (\neg(p \rightarrow q))) \\ &\equiv \neg (\exists y (\neg(\neg p \vee q))) \\ &\equiv \neg \exists y(p \wedge \neg q)\end{aligned}$$
- 变换后语义：不存在这样的课程y，学生202215122选修了y，而学生x没有选y。



带有EXISTS谓词的子查询(续)

```
SELECT DISTINCT Sno  
FROM SC SCX  
WHERE NOT EXISTS
```

//不存在

$\neg \exists y (p \wedge \neg q)$

$\neg (\exists y (\neg (p \rightarrow q)))$

$P \delta \text{ not } (Q)$
 $p \wedge \neg q$

```
(SELECT *
```

```
FROM SC SCY //有一门课程
```

```
WHERE SCY.Sno = '202215122' AND
```

```
NOT EXISTS // 该同学没学过
```

```
(SELECT *
```

```
FROM SC SCZ
```

```
WHERE SCZ.Sno=SCX.Sno AND
```

```
SCZ.Cno=SCY.Cno));
```

SCX

Sno	Cno
202215121	1
202215126	1
202215121	2
202215122	2
202215121	3
202215122	3

SCY

Sno	Cno
202215121	1
202215126	1
202215121	2
202215122	2
202215121	3
202215122	3

SCZ

Sno	Cno
202215121	1
202215126	1
202215121	2
202215122	2
202215121	3
202215122	3

Sno
202215121
202215122

关系代数?



3.4 查 询

3.4.1 单表查询

3.4.2 连接查询

3.4.3 嵌套查询

3.4.4 集合查询

3.4.5 基于派生表的查询

3.4.6 **SELECT** 语句的一般格式



集合查询

- 并操作UNION
- 交操作INTERSECT
- 差操作EXCEPT

注:

- 参加集合操作的各查询结果的列数必须相同；对应项的数据类型也必须相同。
- 标准SQL 没有交和差操作



并操作

[例41] 查询计算机科学系的学生及年龄不大于19岁的学生。

方法一：

```
SELECT * FROM Student WHERE Sdept= 'CS'  
UNION  
SELECT * FROM Student WHERE Sage<=19;
```

- UNION: 将多个查询结果合并起来时，系统自动去掉重复元组。
- UNION ALL: 将多个查询结果合并起来时，保留重复元组

方法二：

```
SELECT * FROM Student  
WHERE Sdept= 'CS' OR Sage<=19;
```



交操作

[例42] 查询同时选修了课程1和课程3的学生学号。

方法一：集合查询

```
SELECT Sno FROM SC WHERE Cno=' 1 '  
INTERSECT  
SELECT Sno FROM SC WHERE Cno=' 3 ';
```

方法二：嵌套查询

```
SELECT Sno FROM SC WHERE Cno=' 1 ' AND Sno IN  
(SELECT Sno FROM SC WHERE Cno=' 3 ');
```



差操作

[例43] 查询没有选修了课程1的学生学号。

方法一：集合查询

```
SELECT Sno FROM student  
EXCEPT  
SELECT Sno FROM SC WHERE Cno = '1';
```

方法二：嵌套查询

```
SELECT Sno FROM student WHERE NOT EXISTS  
(SELECT * FROM SC WHERE SC.Sno=student.Sno  
and Cno='1');
```



3.4 查 询

3.4.1 单表查询

3.4.2 连接查询

3.4.3 嵌套查询

3.4.4 集合查询

3.4.5 基于派生表的查询

3.4.6 SELECT 语句的一般格式



基于派生表的查询

子查询不但能够出现在Where子句中，也能够出现在from子句中，作为一个临时表成为主查询的对象。

- [例44] 找出每个学生超过他自己选修课程平均成绩的课程号

方法一: **Select Sno, Cno From SC x**
Where Grade >= (select avg(Grade)
from SC y where y.Sno=x.Sno);

方法二:

Select Sno, Cno from SC, (select sno,avg(Grade)
from SC group by sno)
AS Avg_sc(avg_sno,avg_grade)
where sc.Sno=Avg_sc.avg_sno and
sc.grade>=Avg_sc.avg_grade;



3.4 查 询

3.4.1 单表查询

3.4.2 连接查询

3.4.3 嵌套查询

3.4.4 集合查询

3.4.5 基于派生表的查询

3.4.6 SELECT 语句的一般格式



3.4.6 SELECT语句的一般格式

SELECT [ALL|DISTINCT]

<目标列表表达式> [别名] [, <目标列表表达式> [别名]] ...

FROM <表名或视图名> [别名]

[, <表名或视图名> [别名]] ...

[**WHERE** <条件表达式>]

[**GROUP BY** <列名1>[, <列名1'>] ...

[**HAVING** <条件表达式>]

[**ORDER BY** <列名2> [ASC|DESC] [, <列名2'> [ASC|DESC]] ...];



典型DBMS介绍—MySQL

■ MySQL

■ MySQL安装

- 官网下载，链接 <https://www.mysql.com/downloads/>
- Download MySQL Community Server （版本8.0版本）



典型DBMS介绍—OpenGauss

■ OpenGauss

参见: <https://www.opengauss.org/zh/>



To be continued
Practice makes perfect!



厚德

笃学

崇实

尚新

下课了。。



休息。。。

